第2课

C++ 面向对象与资源管理

第二课 · O01–O40

AIM Robotics

2026–2027 新人培训 · C++17

承接第一课的值、函数、引用和指针知识。逐步建立对象创建、接口、生命周期、多态与资源管理的完整模型。

本讲义包含概念介绍和本课全部练习题面。代码解释片段用于说明当前概念；题目“初始代码”需要独立补全后运行。

阅读顺序

2.1 类、对象与封装 3

2.2 构造与成员初始化 7

2.3 对象的读取、修改与传递 12

2.4 对象组合与协作 18

2.5 生命周期、复制与资源管理 22

2.6 继承与基类部分 27

2.7 虚函数、多态与对象切片 31

2.8 动态对象、所有权与异常清理 34

练习与阅读

每个小节先介绍概念，再列出对应练习。题号沿用 CPP-Classroom，题目按本课的学习顺序出现。样例没有输入时标为“无输入”，其含义与平台上的空样例输入相同。

概念代码片段只展示当前需要解释的部分。运行某道题时，以该题完整初始代码为起点，保留所需头文件与唯一的 main。讲义不在题面前附上整题参考答案。

2.1 类、对象与封装

第一课的结构体把字段组织在一起。本课进一步把操作也放进类型，让每个对象维护自己的状态和使用规则。

本节题号：O01、O02、O03、O04、O05

先建立对象模型

类型说明一类数据有哪些成员，以及可以进行哪些操作。对象是这个类型的一份具体实例。即使两个对象属于同一个类型，它们的普通数据成员也分别保存状态。对某个对象调用成员函数时，函数操作的是这个对象的成员，而不会自动修改所有同类型对象。Python 是否学过类不影响理解这里的模型，只需要接上第一课的记录与函数。

```
class Counter {
public:
    void tick() { ++count_; }
    int value() const { return count_; }
private:
    int count_ = 0;
};
```

这段类型把“计数值”和“计数加一”放在一起。Counter a 创建一个对象，a.tick() 对 a 调用操作，a.value() 取得它的当前计数。这里 const 成员函数先读成“这个操作只读当前对象”，第 2.3 节会正式解释。类定义末尾和结构体一样，需要分号。

配套练习

O01 第一个类

源文件：O01.cpp

定义 Robot 含公开 id 与 greet()，创建 id=7 对象并调用 greet 输出 Robot 7。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	Robot 7

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

O02 两个对象的状态

源文件：O02.cpp

创建两个 Counter，只给第一个调用两次 tick，输出两个计数。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	2 0

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

访问权限建立接口边界

public 部分向调用方开放，private 部分只允许类自身成员及被明确授予权限的代码访问。对外调用者不能直接改写私有字段，因此类型可以在成员函数里统一验证修改是否合法。把字段藏起来只是手段，目标是确保对象始终满足它承诺的规则。读取状态时通过 getter 访问，修改状态时通过有语义的操作。

名字末尾的下划线只是本课示例用来识别成员的命名习惯，不是 private 的语法。访问权限由 public、private 等声明决定。struct 默认成员公开，class 默认成员私有，它们都能定义数据成员、成员函数和构造函数，不能把两者理解为完全不同的两套对象机制。

配套练习

O03 private 访问边界

源文件：O03.cpp

Motor 的 speed_ 私有，set_speed 把请求夹到 [-3,3]，getter 返回速度。分别请求 9、-9 并输出。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	3 -3

初始代码

```
#include <iostream>
#include <algorithm>

int main() {
    // TODO
    return 0;
}
```

O05 struct 与 class

源文件：O05.cpp

定义 struct Point 和 class Hidden，各自含 int x，Point 直接访问，Hidden 通过 getter 访问，都输出 3。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	3 3

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

失败操作也需要规定状态

一个修改接口可能接受、限幅或拒绝请求。限幅会把输入调整到允许区间，再写入成员；拒绝则可能要求状态完全保持。先检查全部前提，再提交状态变化，可以避免一部分字段已经变了，另一部分才发现失败。返回 bool 时，调用方要理解它说明的是操作是否成功，而不是简单的某个成员数值。

同一个类型的多个对象应该能独立遵守这些规则。检查方法是对一个实例调用操作，再查看另一个实例是否仍保留原状态。初学面向对象时，不需要先讨论复杂设计模式，把“实例各自有状态”和“公开接口保护规则”理解清楚已经足以完成本节练习。

配套练习

O04 拒绝无效修改

源文件：O04.cpp

Magazine 初值 5，reload(n) 仅接受 n>=0 且总数≤10，成功返回 true，失败保持原状态。测试 +4 与 +2。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	`1 0 9`

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

2.2 构造与成员初始化

一个对象刚出现时，就应该有明确而合法的状态。构造函数负责把这种保证建立起来。

本节题号：O06、O07、O08、O09、O10

构造函数参与对象创建

构造函数名字与类名相同，没有返回类型。对象创建时，编译器根据给出的参数选择合适构造函数。无参数的创建需要可用的默认构造函数；一旦声明了其他构造函数，不能再想当然地假设编译器一定仍会提供无参数版本。构造函数应把输入转换为对象可接受的初始状态。

Python __init__ 常被用于初始化实例，但它与 C++ 构造过程不能一一等同。C++ 的基类和成员对象要在构造函数体执行前完成初始化，类型、顺序和可用构造函数都会受到编译规则检查。先理解这个时间顺序，比只记住两种语言的函数名更重要。

初始化列表先于函数体

```
class Sensor {
public:
    explicit Sensor(int id) : id_(id) {}
    int id() const { return id_; }
private:
    const int id_;
};
```

冒号之后的 id_(id) 是成员初始化列表的一项。它用参数初始化 id_，然后才进入构造函数体。若先让成员默认初始化，再在函数体里赋值，语义就是“创建后再修改”。对于 const 成员，创建之后不能重新赋值，必须在初始化阶段给值。引用成员和没有默认构造函数的成员也需要类似地在创建时安排好。

配套练习

O06 成员初始化列表

源文件：O06.cpp

Battery 构造函数接收 capacity 和 initial，容量至少 1，电量夹到 [0,capacity]，构造 (10,20) 输出电量。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	10

初始代码

```
#include <iostream>
#include <algorithm>

int main() {
    // TODO
    return 0;
}
```

O07 const 数据成员

源文件：O07.cpp

Sensor 具有 const int id_，构造时初始化为 7，getter 输出编号。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	7

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

同一类型的不同创建方式

委托构造让一个构造函数选择同类另一个构造函数完成基础初始化，以免默认值和合法性规则写出多个版本。例如默认构造可以把工作委托给接收参数的构造函数。委托之后才执行当前构造函数体，不能一边委托一边在同一个初始化列表里单独初始化其他成员。

单参数构造可能参与从其他类型到本类型的隐式转换。explicit 限制这类隐式使用，使调用处明确表达“我要构造这个类型”。它不妨碍普通的显式直接构造。读取接口若接受 const Distance&，调用者可以先明确创建 Distance，再传入引用。

```
Timer() : Timer(default_interval) {}
explicit Distance(double meters);
```

配套练习

O08 委托构造函数

源文件：O08.cpp

Timer() 默认 100ms，Timer(int) 接受正值，非正值变为 1。创建默认对象和 Timer(0)，输出各间隔。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	`100 1`

初始代码

```
#include <iostream>
#include <algorithm>

int main() {
    // TODO
    return 0;
}
```

O09 explicit 防止隐式转换

源文件：O09.cpp

Distance 由米数构造，显式创建 Distance(3)，print_distance 接受 const Distance& 并输出值。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	3

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

成员对象必须先被构造

如果 Robot 拥有一个 Motor 成员，且 Motor 只有需要编号的构造函数，那么 Robot 创建时必须通过初始化列表把这个编号交给 Motor。进入 Robot 构造函数体之前，该成员已经需要完成构造，不能到函数体里才尝试“补上创建”。

成员初始化的实际顺序按成员在类中的声明顺序决定，不按初始化列表的书写顺序。良好习惯是把初始化列表也按声明顺序写，减少误读。第二课后面引入继承时，基类部分还会先于成员完成初始化，这条时间线会继续延伸。

配套练习

O10 无默认构造的成员

源文件：O10.cpp

Robot 持有 Motor，Motor 只接受 id 构造，Robot 必须初始化它；构造 Robot(4) 输出电机 id。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	4

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

2.3 对象的读取、修改与传递

第一课已经区分副本、引用和指针。把这些机制放到类的接口里，就能看懂 const 方法、this、链式调用和对象容器。

本节题号：O11、O12、O13、O14、O15、O16、O17、O19、O40

只读对象与只读成员函数

成员函数参数列表之后的 const 修饰这个成员函数对当前对象的访问方式。读取计数、编号或状态的接口通常应声明为 const，使 const 对象也能调用它。普通非 const 成员函数可以修改对象，不能通过一个 const 对象直接调用。const 方法主要保护对象自身状态，不意味着它完全没有外部副作用，也不保证它引用的所有外部对象都绝不会改变。

第一课使用过 values.size() 和 input.str() 这样的成员调用，本课只是在已有使用经验上解释接口限制。设计 getter 时，既考虑返回的类型，也考虑函数是否应该是 const，让调用方无需为读取状态而获得修改权限。

配套练习

O11 const 成员函数

源文件：O11.cpp

Counter 提供 value() const，const Counter(8) 能读取计数，不能调用 tick。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	8

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

this 标识当前对象

普通非静态成员函数中，this 是指向当前接收对象的指针。this->x 访问当前对象的成员，与 (*this).x 对应。参数也叫 x 时，this->x = x 可以把左侧成员与右侧参数明确区分。this 本身指针身份不能重新赋成另一个对象。const 成员函数里的 this 提供对 const 对象的访问。

成员函数若返回当前对象的引用，就可以写 return *this。调用方拿到的仍是同一对象，所以还能继续调用另一个设置接口，形成链式调用。返回引用不会创建一个自动延寿的新对象，原对象必须在整段使用期间仍然有效。

配套练习

O12 this 与同名参数

源文件：O12.cpp

Pose 的 set_x(int x) 用 this->x=x 区分同名变量，设置 5 并输出。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	5

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

O13 链式调用返回引用

源文件：O13.cpp

Config 的 set_speed 与 set_id 返回 Config&，链式设置 3、7 后输出 id:speed。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	7:3

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

返回副本与返回借用

返回容器值会给调用方一个独立的容器结果；返回 const 引用则提供对内部现有容器的只读访问。调用方修改副本不会影响原容器，但保存内部引用会受到原对象生命周期和某些容器操作的影响。不能把 getter 返回引用理解为“永远有效”，也不能为了避免一次可能的复制而忽略接口的使用边界。

配套练习

O14 引用访问与值访问

源文件：O14.cpp

Buffer 包装 vector，copy() 返回副本，view() 返回 const 引用。修改副本首项为 9，输出 view 首项和副本首项。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	1 9

初始代码

```
#include <iostream>
#include <vector>

int main() {
    // TODO
    return 0;
}
```

静态成员函数与对象复制

static 成员函数属于类型，不依赖某个实例，因此没有 this，不能直接读取普通实例成员。它适合判断一个参数是否合法这类不需要对象状态的工具操作。通过 Limits::valid 调用时，:: 说明访问类型中的成员。

若类没有自定义特殊复制行为，成员通常依照各自类型进行复制。一个类含 vector<int> 时，复制对象会复制这个容器的值，修改副本的整数元素不会影响原对象。若成员是普通指针，复制的则是地址值，这两种情况要分开。函数按值接收对象时创建参数副本，按引用接收时使用原对象。

配套练习

O15 静态成员函数

源文件：O15.cpp

Limits::valid 不依赖对象，只判断速度是否在 [-3,3]，分别检查 3、4。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	`1 0`

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

O16 对象值复制

源文件：O16.cpp

Robot 内含 vector{1,2}，复制 a 到 b，修改 b 首项为 9，输出 a 与 b 首项。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	`1 9`

初始代码

```
#include <iostream>
#include <vector>

int main() {
    // TODO
    return 0;
}
```

O17 引用传参与复制传参

源文件：O17.cpp

Counter 从 0 开始，touch_copy 按值加一，touch_ref 按引用加一，调用二者后输出原计数。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	1

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

对象集合和类外定义

vector<Unit> 保存的是 Unit 对象。emplace_back 把参数交给元素构造过程，在容器末尾创建元素。使用 const Unit& 遍历可以读取对象而不按轮复制。容器增长时可能重新分配存储，使先前保存的元素指针或引用失效，因此不要把元素的地址当作永远固定。

成员函数可以在类中声明，在类外用 类型名::成员名 给出定义。这样接口与实现更易分开阅读。O40 仍然要求整个程序保留在单一 .cpp 中；类外定义并不等同于必须拆成多个文件。

配套练习

O19 vector 保存对象

源文件：O19.cpp

Unit(id) 构造对象，把 2、7 两对象 emplace_back 到 vector，按 const 引用遍历输出 id。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	2 7

初始代码

```
#include <iostream>
#include <vector>

int main() {
    // TODO
    return 0;
}
```

O40 声明与类外定义

源文件：O40.cpp

把 Motor 声明与成员定义分开，但仍保留在同一个 .cpp 中。set 限幅到 [-3,3]，设置 8 后输出 3。多文件拆分仅作为讲师本地工程扩展。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	3

初始代码

```
#include <iostream>
#include <algorithm>

int main() {
    // TODO
    return 0;
}
```

2.4 对象组合与协作

许多功能可以通过多个小对象协作完成。组合把部件作为成员，让外层对象组织步骤而不是包办所有细节。

本节题号：O21、O22、O23、O24、O25

拥有成员对象

Chassis 可以有左、右两个 Motor 成员，Robot 可以有 Battery 成员。这是一种“拥有部件”的关系，每个成员负责自己的状态与接口。外层对象构造时准备成员，运行时调用成员的公开操作。外层对象的生命周期也包含这些成员的构造和销毁。

组合关系先从实际需求判断：底盘拥有电机，而底盘不是一种电机。这里不需要为了复用接口而使用继承。把部件的规则放到各自类型内部，再由组合对象协调，可以减少多个地方各自修改同一份状态的机会。

配套练习

O21 组合两个电机

源文件：O21.cpp

Chassis 包含左右两个 Motor，每个 set 接受速度，drive(2,3) 后输出左右速度。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	2 3

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

协调过程需要清楚的顺序

一个动作可能要求先确认电量足够，再消耗电量，再改变位置。若校验失败，就不能只执行其中一部分修改。接口应说明动作是否完成，并保证外部能观察到的状态一致。成员对象也可以通过返回值报告操作结果，外层对象依此决定是否继续。

配套练习

O22 电池与动作组合

源文件：O22.cpp

Robot 持有电量 2 的 Battery，每次 step 消耗 1，仅足够时 x 加一，调用三次后输出 x 与剩余电量。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	`2 0`

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

配置是对象行为的输入

把配置通过构造参数传入，能让不同对象在同一套实现下采用不同限制。若构造时复制配置，之后调用方修改自己的那份配置，不会自动改变已经创建对象的配置副本。这一点仍然来自第一课的值复制模型。若以后需要共享实时配置，则必须明确选择引用或其他共享机制，本题不引申到那种设计。

配套练习

O23 注入可替换配置

源文件：O23.cpp

Policy 含 max_speed，Controller 通过构造复制配置，对请求限幅。两个配置 2 和 5 对请求 4 各输出结果。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	2 4

初始代码

```
#include <iostream>
#include <algorithm>

int main() {
    // TODO
    return 0;
}
```

快照与内部集合

快照把某个时刻的状态按值返回。调用方可以修改自己的快照，不会反过来修改原对象。它适合让外部观察状态，同时保留内部修改入口。快照和内部引用的语义不同：前者是独立记录，后者仍借用内部对象。

类内部也可以维护 vector<string> 等集合，公开接口决定哪些消息被接收、如何汇报，以及读取是否改变状态。添加时统一过滤空串，输出时统一保持顺序，这些规则应由接口实现，而不依赖每个调用者都记得过滤。组合本身没有额外的神秘语法，它依靠成员对象、构造和普通成员调用完成协作。

配套练习

O24 只读状态快照

源文件：O24.cpp

Robot::snapshot 返回 State 值，拿到快照后修改其 x=99，原对象 x 仍为 3。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	3 99

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

O25 诊断消息集合

源文件：O25.cpp

Diagnostics 保存非空消息，add 忽略空串，report 每行输出一条，依次添加 OK、空串、LOW。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	OK LOW

初始代码

```
#include <iostream>
#include <string>
#include <vector>

int main() {
    // TODO
    return 0;
}
```

2.5 生命周期、复制与资源管理

对象何时构造、何时结束，会决定它管理的数据或资源何时可用。析构把结束时需要的工作放进类型本身。

本节题号：O18、O20、O37、O39

构造与析构的时间线

普通局部对象在执行到相应声明时构造，在离开所属作用域时析构。同一作用域内，已经成功构造的普通局部对象通常按完成构造的相反顺序销毁。析构函数名写成 ~类型名，没有返回值和参数。它适合在对象结束时完成清理。不要为仍在正常使用的局部对象手动调用析构函数，否则之后自动销毁时可能破坏生命周期规则。

执行位置	对象状态
进入外层并创建 a	a 存活
进入内层并创建 b	a、b 都存活
离开内层	b 析构，a 继续存活
离开外层	a 析构

配套练习

O18 作用域销毁顺序

源文件：O18.cpp

Tracer 构造输出 +名称，析构输出 -名称。外层 a，内层 b，随后结束 main，预测全部输出。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	+a +b -b -a

初始代码

```
#include <iostream>
#include <string>
#include <utility>

int main() {
    // TODO
    return 0;
}
```

静态数据与复制构造

静态数据成员由类型的各个实例共同访问，不是每个对象分别拥有一份。C++17 的 inline static 可以在类内给出定义和初值。活对象计数需要在每次对象成功构造时增加，在析构时减少。复制构造创建新对象，因此也需要增加计数。

用一个已有对象初始化另一个新对象，涉及复制构造；把值赋给一个已经存在的对象，涉及赋值操作。两者不能混为一谈。默认复制一个普通计数成员不会自动维护全局活对象计数，所以 Tracker 需要在复制构造函数中明确完成这项责任。

```
Tracker(const Tracker& other);
Tracker& operator=(const Tracker& other);
~Tracker();
```

上面依次是复制构造、复制赋值和析构的接口形式。复制构造没有返回类型；operator= 命名赋值操作，常返回当前对象的引用。这里只识别这些特殊成员的职责和语法，不展开一般运算符重载。

配套练习

O20 静态活对象计数

源文件：O20.cpp

Tracker 的默认构造和复制构造使 live+1，析构 live-1，作用域外最终计数为 0。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	2 0

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

资源与生命周期绑定

资源不仅指内存，也可以是文件句柄、锁或其他需要成对获取和释放的东西。RAII 的做法是由对象承担资源管理责任：创建时建立资源，结束时释放资源。调用者依照对象的生命周期使用资源，减少多个返回分支都要手动清理的负担。这里先理解责任和时机，异常路径将在第 2.8 节进一步说明。

std::string、std::vector 等标准库类型已经管理自己的存储。当一个 Log 只含 vector<string>，且没有额外的手动资源责任时，通常可以依靠成员完成复制和析构，不必自己再编写一组特殊成员函数。这就是 Rule of Zero 在本课中的意义：选用能管理资源的成员，让默认行为正确组合。

配套练习

O37 Rule of Zero

源文件：O37.cpp

Log 包装 vector<string>，使用默认复制、析构，复制后只向 b 追加，输出 a 与 b 长度。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	1 2

初始代码

```
#include <iostream>
#include <string>
#include <vector>
#include <utility>
#include <cstddef>

int main() {
    // TODO
    return 0;
}
```

有些对象就不应该被复制

某个设备对象可能代表唯一的使用权，把它复制成两份会让管理责任变得不清楚。可以用 = delete 明确禁止复制构造与复制赋值，让非法复制在编译时被拒绝。= default 表示请求编译器提供相应默认行为。删除复制不等于自动获得移动能力，本题只要求理解可默认创建、可使用，但不可复制的接口。

对初学者来说，重要的不是背下所有特殊成员函数，而是能判断这个类型复制后意味着什么。若成员的默认行为已经正确，就依靠成员；若资源责任要求唯一，就把限制明确写在类型接口里。普通指针自身不负责销毁它指向的对象，这也是下一节以后讨论所有权时必须保留的区别。

```
Device() = default;
Device(const Device&) = delete;
Device& operator=(const Device&) = delete;
```

配套练习

O39 不可复制的设备

源文件：O39.cpp

Device 删除复制构造与复制赋值，仍可默认构造和调用 id 输出 7。指出复制语句应编译失败。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	7

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

2.6 继承与基类部分

组合已经说明“拥有一个部件”。继承用来描述一种更具体的类型与其基类之间的关系。

本节题号：O26、O27、O28、O29、O30

派生对象包含基类部分

class Derived : public Base 声明公有继承。一个 Derived 对象包含 Base 部分，也可以增加自己的成员和操作。把 Derived 用在需要合适 Base 引用或指针的位置，表达的是它也具有基类接口。继承应与组合区分：拥有电池适合成员组合，一种更具体的传感器与统一传感器接口则可能适合继承。

公有继承不会让基类的 private 成员变成派生类可以直接访问的字段。派生类使用基类公开的接口完成这些访问，或者在设计允许时通过 protected 成员实现受控扩展。不要为了省去 getter 就把所有状态改成 protected。

配套练习

O26 public 继承

源文件：O26.cpp

Robot 提供 id=7，Sentry 公有继承 Robot 并增加 patrol，输出 id 和 PATROL。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	7 PATROL

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

先创建基类，再创建成员

派生构造函数的初始化列表可以给基类构造函数传参。基类没有可用默认构造时，派生类必须选择合适的带参数构造。这里给基类传参属于基类部分的初始化，与第 2.2 节同一类型内的委托构造需要区分。完整对象的初始化顺序先处理基类，再按声明顺序处理成员，最后执行派生构造函数体。析构的对应顺序相反，派生析构函数体执行后，成员和基类部分继续完成销毁。

配套练习

O27 基类构造委托

源文件：O27.cpp

Robot 必须给定 id 构造，Sentry 接收 id 并初始化基类，构造 9 输出 9。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	9

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

public、protected 与 private

成员访问	类外调用方	派生类成员函数
public	可以使用	可以使用
protected	不可直接使用	可在符合规则的派生访问中使用
private	不可直接使用	不能直接访问基类私有成员

这里的表格针对最基本的访问场景。protected 允许派生实现访问基类的受保护状态，但它不是“所有继承相关对象都任意可访问”的许可。当前题目通过派生对象修改继承而来的 hp，再提供公开读取接口，就足以理解用途。

配套练习

O28 protected 的可见性

源文件：O28.cpp

Base 的 hp=10 为 protected，Derived::damage 减 3，通过 public getter 输出 7。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	7

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

O29 构造与析构的层次

源文件：O29.cpp

Base 与 Derived 构造、析构分别输出 +B、+D、-D、-B，创建一个 Derived 验证次序。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	+B +D -D -B

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

同名隐藏尚未构成动态多态

派生类声明一个与基类同名的非虚函数时，直接通过派生对象查找这个名字，可能选到派生版本；使用对象.Base::name() 则明确请求基类版本。对于非虚函数，通过基类接口调用不会自动按实际派生类型重新选择实现。Python 的方法查找直觉不能原样套用，下一节将引入显式的 virtual 机制。

配套练习

O30 同名隐藏与限定调用

源文件：O30.cpp

Base::name 返回 BASE，Derived 定义同名非虚 name 返回 DERIVED，分别调用 d.name 与 d.Base::name。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	DERIVED BASE

初始代码

```
#include <iostream>
#include <string>

int main() {
    // TODO
    return 0;
}
```

2.7 虚函数、多态与对象切片

调用方有时只关心统一接口，实际行为则由具体对象决定。虚函数让这件事成为 C++ 类型接口的明确组成部分。

本节题号：O31、O32、O33

基类接口与实际对象

基类把某个成员函数声明为 virtual，派生类提供匹配的覆盖实现。通过指向实际派生对象的基类引用或指针调用该虚函数时，程序按实际对象选择最终覆盖函数。override 请求编译器检查这里确实覆盖了某个基类虚函数，能帮助发现参数、const 等签名不匹配的问题。

```
struct Base {
    virtual int value() const;
};
struct Derived : Base {
    int value() const override;
};
```

Python 常见的运行时方法选择让这种“同一接口，不同行为”比较容易理解。C++ 仍要求我们区分是否为虚函数，以及当前保存的是完整派生对象还是一个基类值。virtual 不是装饰性关键词，它决定通过基类接口调用时是否发生这里讨论的动态派发。

配套练习

O31 virtual 动态派发

源文件：O31.cpp

Base::kind 虚函数返回 BASE，Derived override 返回 SENTRY，通过 const Base& 调用输出 SENTRY。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	SENTRY

初始代码

```
#include <iostream>
#include <string>

int main() {
    // TODO
    return 0;
}
```

纯虚函数表达需要实现的接口

在虚函数声明后写 = 0 可以声明纯虚函数。具有未实现纯虚接口的抽象类不能直接创建实例，但可以通过它的引用或指针接收具体对象。调用方依赖 read() 这样的统一约定，具体类型负责提供结果。这使调用代码可以只关心接口，不必在每次调用时手动判断具体类型。

纯虚接口可以只有最小操作，不需要在第一份设计里穷举所有未来功能。当前题目关注 const 读取接口、具体派生实现与基类引用传参，按这条最短路径理解即可。保存动态对象和统一销毁的责任将在下一节补齐。

配套练习

O32 纯虚接口

源文件：O32.cpp

定义抽象 Sensor::read() const，FixedSensor 返回 42，print_sensor 接受 const Sensor& 并输出 read。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	42

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

按值复制到基类会切掉派生部分

Base value = derived 创建的是一个真正的 Base 对象，只用派生对象的基类部分初始化它。派生部分不会保存进这个较小的基类值里，这叫对象切片。之后即使通过虚函数调用，value 的实际类型也是 Base，不会凭空恢复已经丢失的派生部分。

保存方式	实际保存的东西	后续虚调用
Base value = derived	独立的 Base 对象	按这个 Base 对象派发
const Base& ref = derived	借用原来的完整派生对象	可派发到派生实现

想保留多态行为时，通常使用合适的引用或指针访问现有对象，或使用能够拥有多态对象的智能指针。引用和普通指针不自动延长目标生命周期。只把签名改成引用，却让目标先结束生命，仍然是不正确的接口使用。

配套练习

O33 对象切片

源文件：O33.cpp

Base::kind 返回 BASE，Derived 返回 DERIVED。Derived d 复制到 Base value=d 后分别输出 value 与基类引用的 kind。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	`BASE DERIVED`

初始代码

```
#include <iostream>
#include <string>

int main() {
    // TODO
    return 0;
}
```

2.8 动态对象、所有权与异常清理

普通指针能提供访问入口，析构能完成对象清理。本节把两者连接起来，明确动态对象由谁负责结束生命。

本节题号：O36、O34、O35、O38

作用域与动态存储期

普通局部对象通常具有自动存储期，作用域退出时自动销毁。动态分配创建的对象则需要管理者在合适时机销毁。常见实现把自动局部存储与动态分配分别联系到栈和堆，但这只是理解实现的常见示意，不是标准对所有对象物理位置的绝对保证。决定清理责任的是对象与所有者的语义。

new 可以创建动态对象并返回指针，delete 可以销毁匹配的对象并释放相应存储。原始指针变量离开作用域时不会自动替你 delete 目标。手动管理很容易在多个返回分支中漏掉释放，也可能重复释放。题库因此采用标准库的独占智能指针来表达这项责任，不要求你实现一套手写内存管理。

独占所有权用类型表达

std::unique_ptr<T> 来自 <memory>，拥有一个对象并在结束时负责删除它。std::make_unique<T>(args...) 创建对象并返回相应所有者。普通指针或引用可以借用目标，但借用者不能比所借对象活得更久，也不应擅自释放它。智能指针本身仍是一个正常的 C++ 对象，因此它能把资源责任接入前面已经学过的析构机制。

unique_ptr 不可复制，因为复制会造成两份独占所有者。std::move 来自 <utility>，它本身只是把表达式转换成允许选择移动操作的形式；真正的所有权转移由 unique_ptr 的移动构造或移动赋值完成。对 unique_ptr 完成这种转移后，源指针为空，目标指针接管对象。不能把所有类型的移动后状态都概括成“一定为空”。

```
std::unique_ptr<int> owner = std::make_unique<int>(value);
std::unique_ptr<int> next = std::move(owner);
```

配套练习

O36 独占所有权转移

源文件：O36.cpp

make_unique 创建值为 7 的整数，将 a 移动给 b，输出 a 是否为空与 *b。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	`true 7`

初始代码

```
#include <iostream>
#include <utility>
#include <memory>

int main() {
    // TODO
    return 0;
}
```

通过基类管理派生对象

当 unique_ptr<Base> 拥有一个 Derived 对象，结束时会通过基类类型删除。用于这种多态删除的基类需要合适的虚析构函数，才能按预期执行派生清理，再清理基类部分。一个常见接口是 virtual ~Base() = default。此时派生对象包含的资源责任可以由相应析构过程完成。

析构顺序与第 2.6 节的对象层次保持一致：先执行派生析构函数体，再依次清理成员和基类。智能指针负责触发删除，但它并不会弥补一个缺少正确虚析构约定的多态基类。

配套练习

O34 虚析构函数

源文件：O34.cpp

用 `unique_ptr<Base>` 拥有 Derived，派生析构输出 D、基类析构输出 B。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	D B

初始代码

```
#include <iostream>
#include <memory>

int main() {
    // TODO
    return 0;
}
```

多态对象的集合

vector<unique_ptr<Base>> 的每个元素都持有一个独占所有者，可以分别管理不同具体派生类型。容器保存的不是被切片的 Base 值，而是具有同一基类接口的对象入口和所有权。遍历时通过指针调用虚函数，得到各实际对象的实现。构造派生对象、转换到基类所有者、容器保存和虚调用，这些步骤都已经在前文分别准备好。

向这个容器加入已有 unique_ptr 时，需要转移所有权，不能复制。遍历读取时也不要按值复制 unique_ptr，可以通过 const 引用访问元素，再使用 -> 调用对象接口。容器和元素析构时，各所有者会依次清理自己的目标。

配套练习

O35 多态容器

源文件：O35.cpp

Shape::area 纯虚函数，Square 边长 3，Rectangle 边长 2、4。用 vector<unique_ptr<Shape>> 保存并输出总面积。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	17

初始代码

```
#include <iostream>
#include <vector>
#include <memory>

int main() {
    // TODO
    return 0;
}
```

异常会改变控制流，析构仍承担清理

throw 抛出异常对象，正常执行路径中断，程序寻找匹配的 catch。传播途中离开的作用域里，已经成功构造的自动对象会执行析构，这个过程常称为栈展开。因此 Guard 的生命周期管理在正常离开和异常离开两条路径上都能完成释放。若程序因无人处理的异常直接终止，不能依赖所有清理都如正常路径发生；当前题目提供外层 catch。

```
try {
    Guard guard;
    throw std::runtime_error("failure");
} catch (const std::runtime_error& error) {
    // handle the failure
}
```

runtime_error 需要 <stdexcept>。catch 使用 const 引用可避免不必要的异常对象复制，也保留相应类型信息。本课只需要把异常理解为另一条离开当前作用域的控制路径，不扩展复杂异常体系。资源放进正确的管理对象，才让这条路径也能可靠遵守获取与释放约定。

配套练习

O38 异常与析构清理

源文件：O38.cpp

Guard 构造输出 ACQUIRE，析构输出 RELEASE。在作用域内抛 runtime_error，外层捕获输出 CAUGHT。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	ACQUIRE RELEASE CAUGHT

初始代码

```
#include <iostream>
#include <stdexcept>

int main() {
    // TODO
    return 0;
}
```

两节课连成一套模型

第一课学习类型如何解释值、函数如何传递值，以及引用和指针怎样访问对象。第二课为对象补上创建、接口、结束和资源责任。读到一段 C++ 代码时，可以依次问：这里是什么类型，是否创建副本，正在借用谁，对象还是否存活，最后由谁负责清理。把这些问题与语法位置对应起来，比孤立背关键词更容易迁移到实际工程。

概念资料

本书按 C++17 使用现有题目。以下资料用于核对基本语言与库概念，工作草案网页中的后续标准新增功能不属于本课要求。

Python 官方文档：Execution model
https://docs.python.org/3/reference/executionmodel.html

Python 官方文档：Glossary
https://docs.python.org/3/glossary.html

GCC 官方文档：Overall Options
https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html

C++ 工作草案：Input/output library
https://eel.is/c++draft/input.output

C++ 工作草案：String-based streams
https://eel.is/c++draft/string.streams

C++ 工作草案：References
https://eel.is/c++draft/dcl.ref

C++ 工作草案：Object lifetime
https://eel.is/c++draft/basic.life

C++ Core Guidelines：Resource management
https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#S-resource